

DOCUMENTACION TECNICA

Propuesta: integracion Hermes

ElatoAI ESP32-S3 Zero

PROPUESTA-HERMES-BRIDGE.MD

Propuesta: integracion Hermes — ElatoAI ESP32-S3 Zero

TABLA DE CONTENIDOS

Propuesta: integracion Hermes — ElatoAI ESP32-S3 Zero	2
Descripcion general.....	2
Flujo de operacion principal	3
Arquitectura	3
Archivos involucrados	4
Funcionamiento interno.....	4
Hechos verificados del protocolo (firmware ↔ servidor)	4
Hechos verificados de Hermes.....	5
Decisiones de diseño	6
Riesgos y mitigaciones.....	7
Estado de implementacion	7
Referencia rapida.....	8

Descripcion general

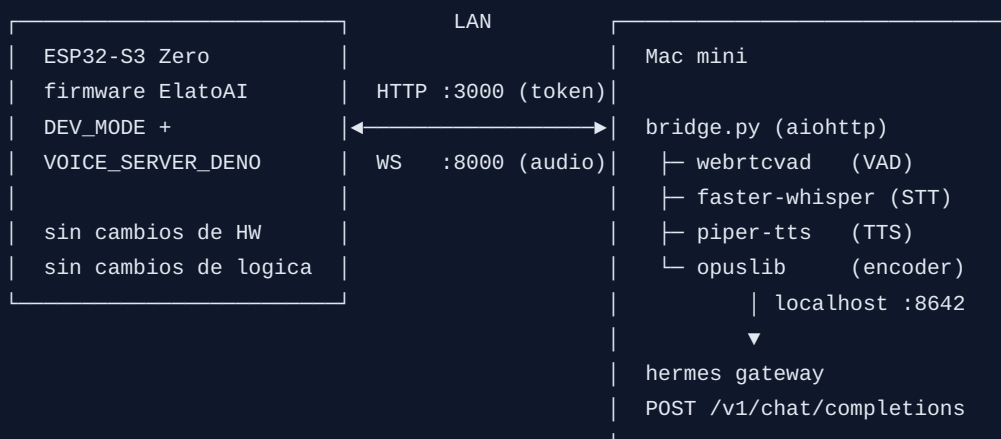
Documento de diseño y contexto de la integracion del hardware ELATO (ESP32-S3 Zero + INMP441 + MAX98357A + KY-004 + WS2812) con el agente **Hermes** (nousresearch/hermes-agent) instalado en una Mac mini de la LAN domestica. El enfoque: un **punto Python** en la Mac mini que imita el protocolo del servidor Deno de ElatoAI, de modo que el firmware solo necesita un cambio de configuracion de 3 lineas (modo `DEV_MODE`) y **cero cambios de hardware ni de logica**. No se usa la nube de Elato, Supabase ni ningun servicio de pago: STT, LLM y TTS corren localmente.

NOTA Este documento existe para que cualquier LLM o agente que analice el proyecto entienda *por que* el puente es como es. Todos los hechos del protocolo estan verificados contra el codigo fuente con referencia archivo:linea. La guia de instalacion paso a paso esta en `[[runbook-hermes-bridge]]`.

Flujo de operacion principal



Arquitectura



Archivos involucrados

Archivo	Rol
server/hermes-bridge/bridge.py	El puente completo (~370 lineas, asyncio)
server/hermes-bridge/requirements.txt	Dependencias Python del puente
server/hermes-bridge/com.elato.hermes-bridge.plist	Plantilla launchd (servicio macOS)
server/hermes-bridge/README.md	Arranque rapido
firmware-arduino/src/Config.h	Cambio de modo: DEV_MODE + VOICE_SERVER_DENO
firmware-arduino/src/Config.cpp	IPs de la Mac mini + fix de certificados en DEV_MODE
docs/runbook-hermes-bridge.md	Guia de instalacion paso a paso

Funcionamiento interno

Hechos verificados del protocolo (firmware ↔ servidor)

Todo lo siguiente se leyo del codigo real; es el contrato que el puente imita:

Hecho	Evidencia
En <code>DEV_MODE</code> el WS es sin TLS (<code>websocket.begin()</code> , no <code>beginSSL</code>)	<code>firmware-arduino/src/Audio.cpp:364-387</code>
El audio del microfono sube como PCM crudo 16 kHz mono 16-bit en frames binarios WS (no Opus: el firmware no tiene encoder)	<code>Audio.cpp:176-201</code> (<code>WebSocketStream::write</code> → <code>sendBIN</code>)
El audio de bajada son paquetes Opus crudos, 24 kHz mono, frames de 120 ms (2880 muestras), bitrate 24000, aplicacion voip	<code>server/deno/utils.ts</code>
Al conectar, el servidor envia <code>{"type":"auth","volume_control":N,"pitch_factor":F,"is_ota":B,"is_reset":B}</code>	<code>server/deno/main.ts:59-67</code>
Mensajes de control <code>{"type":"server","msg":X}</code> : <code>AUDIO.COMMITTED</code> (→ PROCESSING), <code>RESPONSE.CREATED</code> (→ SPEAKING), <code>RESPONSE.COMPLETE</code> / <code>RESPONSE.ERROR</code> (→ LISTENING tras ~1 s), <code>SESSION.END</code> (→ dormir)	<code>server/deno/models/openai.ts</code> + maquina de estados del firmware
El dispositivo arranca en PROCESSING; solo transmite mic en LISTENING ; hay que enviarle <code>RESPONSE.COMPLETE</code> al conectar para activarlo	firmware, maquina de estados
Buffer de audio del ESP32: 10 KB (~200 ms de PCM a 24 kHz) → el emisor debe dosificar los paquetes	<code>firmware-arduino/src/Audio.h:24-25</code>
El firmware no tiene VAD : el servidor decide cuando termino la frase	analisis del pipeline de mic
Token: <code>GET http://IP:3000/api/generate_auth_token?macAddress=...</code> → <code>{"token":"..."}</code> ; se guarda en NVS y se manda como <code>Authorization: Bearer</code> en el WS (junto a <code>X-Device-Mac</code> , <code>X-Wifi-Rssi</code>)	<code>firmware-arduino/src/WifiManager.cpp:16-65</code>
El WS reintenta conexion cada 1 s (<code>setReconnectInterval(1000)</code>)	<code>Audio.cpp</code>
Bug latente en DEV_MODE: <code>FactoryReset.h:11</code> referencia <code>Vercel_CA_cert</code> que solo existia en PROD/ELATO → corregido con un <code>#else</code> que define certs vacios en <code>Config.cpp</code>	<code>firmware-arduino/src/FactoryReset.h:1-30</code>

Hechos verificados de Hermes

Verificados clonando el repo `nousresearch/hermes-agent` y leyendo `website/docs/user-guide/features/api-server.md` :

- Con `API_SERVER_ENABLED=true` + `API_SERVER_KEY` en `~/.hermes/.env` , `hermes gateway` expone `POST /v1/chat/completions` **OpenAI-compatible y stateless** en `http://127.0.0.1:8642` .

- `API_SERVER_HOST` por defecto es `127.0.0.1` (solo localhost).
- Hermes **no tiene endpoints de audio por red**: su STT (faster-whisper) y TTS (Piper/Edge/ElevenLabs/OpenAI) son internos de sus propias interfaces (CLI, Telegram...). Por eso el puente aporta su propio STT/TTS.

Decisiones de diseño

1. `DEV_MODE` en vez de parchear `ELATO_MODE`: `DEV_MODE` ya existe en el firmware exactamente para "servidor local sin TLS". Evita certificados, SNI y todo el problema de TLS con IPs privadas. El cambio queda en 3 defines de `Config.h` + 2 IPs de `Config.cpp`, trivialmente reversible.
2. **Puente protocolo-fiel** en vez de modificar el firmware: el firmware validado no se toca; toda la complejidad vive en Python en la Mac mini, donde es facil de depurar y actualizar.
3. **VAD en el puente (webrtcvad)**: el dispositivo streamea mic continuamente sin marcar fin de frase; el puente corta por silencio (800 ms por defecto, frames de 30 ms = 960 bytes a 16 kHz).
4. **STT/TTS propios del puente** (faster-whisper + Piper): Hermes no expone audio por red; ambos son locales, gratuitos y corren bien en una Mac mini.
5. **Pacing de 1 paquete Opus (120 ms) cada 110 ms**: el buffer del ESP32 es de solo 10 KB; enviar mas rapido lo desborda y corta el audio, enviar mas lento lo vacia. $110 < 120$ mantiene el buffer lleno sin desbordarlo.
6. **Hermes via localhost**: la API de Hermes da acceso a terminal; se mantiene ligada a 127.0.0.1 y solo el puente (misma maquina) la consume, asi la `API_SERVER_KEY` nunca viaja por la LAN.
7. **Token sin validacion real**: en una LAN domestica el endpoint de token devuelve un valor fijo y el WS no lo verifica. Simplifica todo; el riesgo (alguien de la propia LAN hablando con el puente) se asume.
8. **launchd con `KeepAlive`** + reconexion WS de 1 s del firmware = el sistema se recupera solo de caidas del puente o reinicios de la Mac mini.

TIP Si en el futuro se quiere streaming de respuesta (empezar a hablar antes de que Hermes termine), el punto de corte natural es trocear la respuesta por frases y encadenar sintesis Piper por frase. El protocolo del dispositivo ya lo soporta (basta seguir enviando paquetes Opus antes del `RESPONSE.COMplete`).

Riesgos y mitigaciones

Riesgo	Impacto	Mitigacion
Latencia de Hermes (agentic, puede usar herramientas)	Respuestas de varios segundos	<code>model_routes</code> en Hermes hacia un modelo rapido; el LED rojo indica "procesando"
IP de la Mac mini cambia (DHCP)	El ESP32 no conecta	Reserva DHCP en el router (paso 2 del runbook)
CPU de la Mac mini saturada rompe el pacing	Audio entrecortado	Whisper <code>small</code> / <code>base</code> int8; puente asncio ligero
Mac mini apagada	Asistente mudo (reintenta cada 1 s)	launchd <code>RunAtLoad</code> ; rollback a ELATO_MODE si se abandona el setup
API de Hermes expuesta por error	Acceso remoto a la terminal de la Mac	<code>API_SERVER_HOST</code> se deja en 127.0.0.1 (advertido en runbook)

Estado de implementacion

A fecha 2026-07-18:

- ON `bridge.py` escrito y verificado en sintaxis (`py_compile`).
- ON Firmware cambiado a DEV_MODE y **compilado con exito** (`pio run`).
- ON IP de la Mac mini inyectada al compilar via `HERMES_SERVER_IP` .
- OFF Puente instalado en la Mac mini (pendiente: runbook pasos 1-5).
- OFF ESP32 reflasheado (sigue corriendo el firmware ELATO_MODE validado; correcto hasta que el puente este desplegado).

Referencia rapida

PROTOCOLO QUE IMITA EL PUENTE (resumen):

```
GET :3000/api/generate_auth_token?macAddress=M -> {"token":"..."}
WS :8000/ (sin TLS)
servidor -> device : {"type":"auth","volume_control":70,
                      "pitch_factor":1,"is_ota":false,"is_reset":false}
servidor -> device : {"type":"server","msg":"RESPONSE.COMPLETE"} # activa mic
device -> servidor: frames binarios PCM 16 kHz mono s16le
servidor -> device : {"type":"server","msg":"AUDIO.COMMITTED"} # fin frase
servidor -> device : {"type":"server","msg":"RESPONSE.CREATED"} # va a hablar
servidor -> device : paquetes Opus crudos (24 kHz, 120 ms, cada 110 ms)
servidor -> device : {"type":"server","msg":"RESPONSE.COMPLETE"} # turno cerrado
```

PIPELINE: PCM 16k → webrtcvad → faster-whisper → Hermes :8642
→ piper → resample 24k → opus 120ms → pacing 110ms → WS